



Project Azure

Fitness Tracker Dashboard

By: Cedric Urbino

Table of Contents

Introduction.....	3
Architectural Diagram.....	4
Resource Group.....	5
Networking.....	5
Virtual Network.....	6
Load Balancer.....	7
NSGs and outbound rule fix.....	8
Bastion.....	10
LAMP Stack Installation.....	11
Storage.....	13
SAS Token Setup.....	13
Fitdashstorage container for images.....	13
Compute.....	15
Virtual Machines.....	15
Database.....	16
Fitnessdb, tables (Users, Workouts, Meals).....	17
Backup (Recovery Services Vault).....	18
VM and DB backups.....	19
Monitoring and Billing.....	20
Azure Monitor Alerts.....	21
Conclusion.....	24
References.....	24

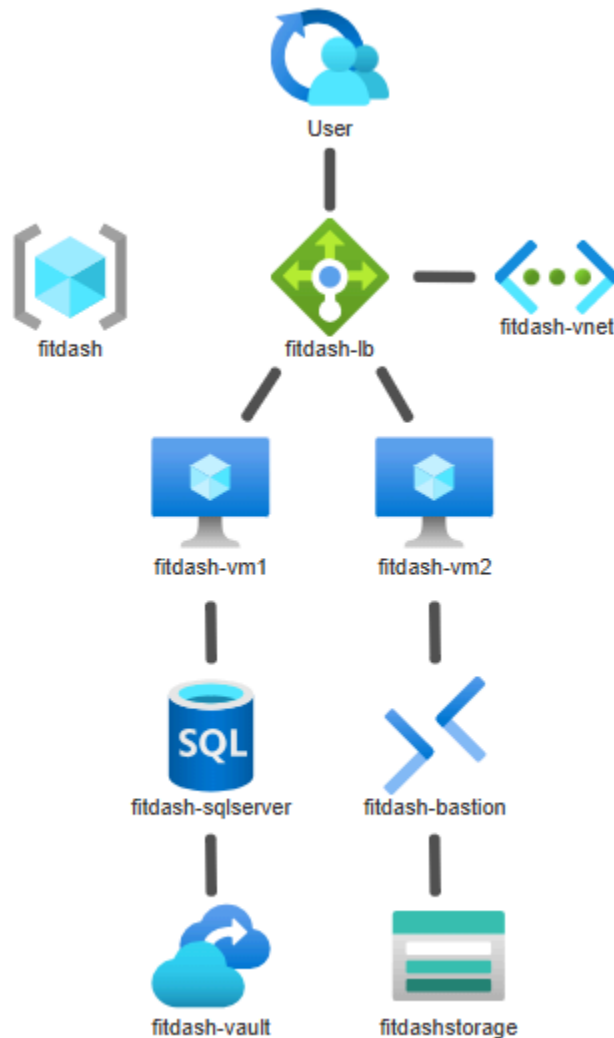
Introduction

For this project, I built a Fitness Tracker Dashboard on Microsoft Azure. The purpose of the project was to simulate a real-world web application that allows users to log their workouts, meals, calories burned, and overall progress toward weight loss goals. I wanted to combine the key cloud services like compute, storage, networking, monitoring, and backup into one integrated solution. The dashboard gives users the ability to record different types of workouts and track the calories burned, enter daily food logs to compare calories consumed versus calories burned, view graphs to visualize progress over time, and see a net calorie balance that would help with their weight loss planning.

This project was also a way to learn how Azure services interact. I hosted the application on virtual machines and made it reliable using a load balancer. A SQL Database handled user data, while Blob Storage stored images like exercise diagrams and food photos. I also configured a backup vault for disaster recovery and set up alerts and a billing alarm to manage costs and ensure availability.

Overall, this project gave me a realistic example of how a cloud-based web application could be deployed, managed, and monitored. It also helped me build confidence with Azure's CLI and portal while working through the challenges of configuration and integration.

Architectural Diagram



The FitDash architecture is designed so all Azure services work together to create a connected and reliable fitness dashboard. Users access the web app through the Azure Load Balancer, which routes their requests evenly between two Ubuntu virtual machines running Apache and PHP. These VMs process workout and meal data, then communicate with the Azure SQL Database to store structured information and Azure Blob Storage for images. The Recovery Services Vault provides daily backups of both the VMs and database, while Azure Monitor and cost alerts track system health and spending. Altogether, these services form a secure, scalable environment where users can log workouts, upload photos, and view their fitness progress in real time.

Resource Group

In order to get started on my project, I first needed to create a resource group that will essentially hold all of the resources that I need to develop my Fitness Tracker Dashboard. I created a resource group named “fitdash” and selected “(US) East US” as the region.

Create a resource group ...

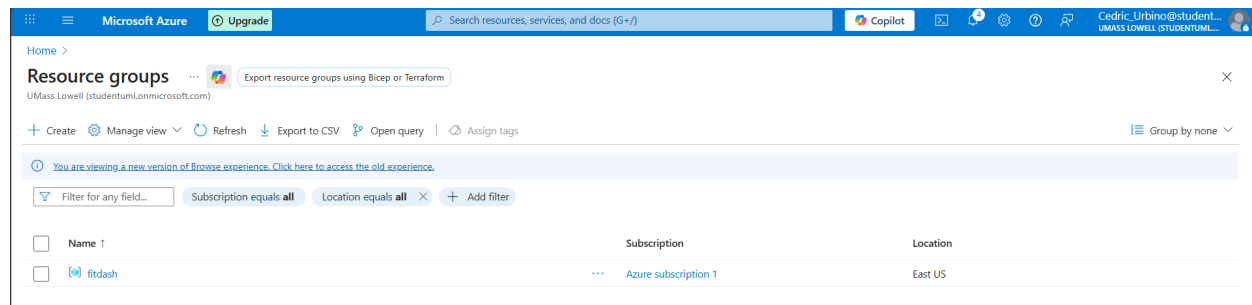
Basics Tags Review + create

Resource group - A container that holds related resources for an Azure solution. The resource group can include all the resources for the solution, or only those resources that you want to manage as a group. You decide how you want to allocate resources to resource groups based on what makes the most sense for your organization. [Learn more](#)

Subscription * ⓘ

Resource group name * ⓘ

Region * ⓘ



When I created my Standard Load Balancer, the VMs lost outbound internet connectivity. This happened because Standard Load Balancers isolate traffic for better security meaning, by default, VMs behind them cannot access the internet unless explicitly allowed. In the below image you can see my configure the frontend IP configuration to my load balancer’s public IP, selected web-subnet, and enabled SNAT

Networking

Networking was one of the trickiest parts of the project. My Load Balancer (fitdash-lb) distributes incoming traffic between both web servers to provide fault tolerance. However, right after I created it, both of my VMs lost internet access. I couldn’t update packages or ping any external addresses. After troubleshooting, I learned that Standard Load Balancers block outbound traffic by default. The solution was to create an Outbound Rule in the Load Balancer configuration, which restored connectivity.

I also created a Virtual Network (VNet) with subnets to separate my web, database, and management layers. This network segmentation made it easier to control access and follow best practices for cloud security. Within the VNet, I used Network Security Groups (NSGs) to allow web traffic on port 80 and deny unnecessary inbound connections.

Finally, I configured Azure DNS to map my public IP to a friendly domain name. This gave the FitDash app a cleaner and more professional web address instead of a numeric IP. Through this networking setup, I learned that even small misconfigurations like missing routes or rules can easily break communication between services. But once it's done correctly, networking becomes the backbone of reliable cloud performance.

Virtual Network

Below shows the settings that I selected to set up and deploy my virtual network. I decided to set up three subnets. The first one is called “web-subnet” with the purpose of hosting my Virtual Machines, this is the subnet that handles the public web traffic through the load balancer. The second subnet is called the “db-subnet” which is used to isolate my database from internet traffic. The last subnet is called “mgmt-subnet” which was made to be reserved for management tools like Azure Bastion letting you log into the VMs without opening their SSH ports.

[Home](#) > [Network foundation](#) | [Virtual networks](#) >

Create virtual network ...

 Validation passed

Basics

Security

IP addresses

Tags

Review + create

[View automation template](#)

Basics

Subscription	Azure subscription 1
Resource Group	fitdash
Name	fitdash-vnet
Region	East US

Security

Azure Bastion	Disabled
Azure Firewall	Disabled
Azure DDoS Network Protection	Disabled

IP addresses

Address space	10.0.0.0/16 (65,536 addresses)
Subnet	default (10.0.0.0/24) (256 addresses)
Subnet	web-subnet (10.0.1.0/24) (256 addresses)
Subnet	db-subnet (10.0.2.0/24) (256 addresses)
Subnet	mgmt-subnet (10.0.3.0/24) (256 addresses)

Microsoft Azure

Upgrade

Search resources, services, and docs (G+/I)

Copilot

Home >

fitdash-vnet-1759617782330 | Overview

Deployment

Search

Delete

Cancel

Redeploy

Download

Refresh

Overview

Inputs

Outputs

Template

✔ Your deployment is complete

Deployment name : fitdash-vnet-1759617782330

Subscription : Azure subscription 1

Resource group : fitdash

Start time : 10/4/2025, 5:43:05 PM

Correlation ID : 58832e9d-015a-42c6-9fb3-5bdf480bc0e3

> Deployment details

< Next steps

Go to resource

Load Balancer

In this project I created a load balancer and labelled it “fitdash-lb” which I used to distribute incoming requests to multiple servers (in my case the fitdash-vm1 and fitdash-vm2).

Home > Load balancing and content delivery | Load balancers >

Create load balancer

✔ Validation passed

Basics

Frontend IP configuration

Backend pools

Inbound rules

Outbound rules

Tags

Review + create

Basics

Subscription

Resource group

Name

Region

SKU

Tier

Type

Azure subscription 1

fitdash

fitdash-lb

East US

Standard

Regional

Public

Frontend IP configuration

Frontend IP configuration name

Frontend IP configuration IP address

LBFrontend

To be created

Backend pools

Backend pool name

LBBBackendPool

Inbound rules

Load balancing rule name

HTTPRule

Create

< Previous

Next >

Download a template for automation

Give feedback

Create load balancer

Basics

Frontend IP configuration

Backend pools

Inbound rules

Outbound rules

Tags

Review + create

A backend pool is a collection of resources to which your load balancer can send traffic. A backend pool can contain virtual machines, virtual machine scale sets, and containers.

+ Add a backend pool

Name	Virtual network	Resource Name	Network interface	IP address	Availability zone	Admin state
▼ LBBBackendPool						
LBBBackendPool	fitdash-vnet	fitdash-vm1	fitdash-vm181	10.0.1.4	-	None
LBBBackendPool	fitdash-vnet	fitdash-vm2	fitdash-vm2400	10.0.1.5	-	None

I set up a health probe on port 80 so the load balancer only distributed traffic to healthy VMs. After setting it up, I lost outbound internet access on my VMs, which I later learned is normal behavior for a Standard Load Balancer.

Home > CreateLoadBalancerBladeV2-20251004183812 | Overview

Your deployment is complete

Deployment name: CreateLoadBalancerBladeV2-20251004183812
Subscription: Azure subscription 1
Resource group: fitdash
Start time: 10/4/2025, 6:57:11 PM
Correlation ID: d1a35454-ac6f-4d7b-ad05-7566e392ce76

Deployment details

Resource	Type	Status	Operation details
NicUpdate-338be824-0293-4bf3-8fd2-f59fc7beeb24-20251004185707	Deployment	OK	Operation details
NicUpdate-559e0ac7-0f95-4c68-9277-e2541317b085-20251004185707	Deployment	OK	Operation details
fitdash-lb	Load balancer	Created	Operation details
fitdash-lb-pip	Public IP address	OK	Operation details

Next steps
[Go to resource](#)

Cost management
Get notified to stay within your budget and prevent unexpected charges on your bill.
[Set up cost alerts >](#)

Microsoft Defender for Cloud
Secure your apps and infrastructure
[Go to Microsoft Defender for Cloud >](#)

Free Microsoft tutorials
[Start learning today >](#)

Work with an expert
Azure experts are service provider partners who can help manage your assets on Azure and be your first line of support.

Home > fitdash-lb | Load balancer

Overview

Resource group: fitdash
Location: East US
Subscription: Azure subscription 1
Subnet ID: 8c187024-9a28-4a71-8b76-29e2893acd73
SKU: Standard
Tier: Regional
Tags: Add tags

Backend pool: LBBackendPool (2 virtual machines)
Load balancing rule: HTTPRule (Tcp/80)
Health probe: HTTPProbe (Http/80)
Inbound NAT rules: -
Outbound rules: -
Frontend IP address: 4.156.98.250 (fitdash-lb-pip)

Configure high availability and scalability for your applications
Create highly-available and scalable applications in minutes by using built-in load balancing for cloud services and virtual machines. Azure

NSGs and outbound rule fix

I used NSGs to allow inbound traffic on ports 80 (HTTP) and 22 (SSH) for web access and remote management while also blocking all the unnecessary inbound connections to protect the VMs from unwanted traffic.

Home > Network foundation | Network security groups

Overview

Showing 1 to 2 of 2 records

Name	Resource group	Location	Subscription	Flow log
fitdash-vm1-nsg	fitdash	East US	Azure subscription 1	
fitdash-vm2-nsg	fitdash	East US	Azure subscription 1	

When I first made the Standard Load Balancer, my VMs lost outbound internet connectivity which happened because they isolate traffic for better security. Meaning by default VMs are left behind without access to the internet unless explicitly allowed. By setting the frontend IP configuration to my Load Balancer's public IP, selecting my web-subnet with my VM, and

enabling SNAT so outgoing traffic from the VMs could be translated to the Load Balancer's public IP allowed my VMs to gain access to the internet for SQL and API communication while still keeping inbound access controlled by the Load Balancer and NSG. This helped me understand how Azure separates internal and public traffic. Setting up networking in Azure taught me that small misconfigurations can quickly affect connectivity across multiple services.

The screenshot shows the 'Add outbound rule' dialog in the Azure portal. The left sidebar displays the navigation menu with 'Outbound rules' selected. The main panel shows the configuration for the 'fitdash-lb' load balancer. The 'Name' field is 'OutboundInternet'. The 'IP version' is set to 'IPv4'. The 'Frontend IP address' is 'LBFrontend (4.156.98.250)'. The 'Protocol' is set to 'All'. The 'Idle timeout (minutes)' is set to '4'. The 'Enable TCP Reset' checkbox is unchecked. The 'Backend pool' is 'LBBackendPool (2 instances)'. The 'Outbound ports' section shows 'Enable default port allocation (not recommended)' is unchecked, 'Choose by' is 'Allocated outbound ports (per instance)', 'Available frontend ports' is '63992', and 'Allocated outbound ports (per instance)' is '0'. The 'Save' button is highlighted in blue.

Add outbound rule

fitdash-lb

Name * OutboundInternet

IP version ☒ IPv4 ☐ IPv6

Frontend IP address * LBFrontend (4.156.98.250)

Protocol * ☒ All ☐ TCP ☐ UDP

Idle timeout (minutes) 4

Enable TCP Reset ☐

Backend pool

This outbound rule applies to all rules in the backend pool. Outbound rules can only be applied to primary IP configuration of a network interface and if the associated backend pool virtual machines don't have an instance-level public IP address. You can go to the respective virtual machine and network interface pages to check IP settings. [Learn more about outbound rules](#)

Backend pool * LBBackendPool (2 instances)

Outbound ports

Azure automatically assigns the number of outbound ports to use for source network address translation (SNAT) based on the number of frontend IP addresses and backend pool instances. [Learn more about outbound connectivity](#)

Enable default port allocation (not recommended) ☐

Choose by * Allocated outbound ports (per instance)

Available frontend ports 63992

Allocated outbound ports (per instance) 0

[Give feedback](#)

[Save](#) [Cancel](#)

The screenshot shows the 'Overview' page for a deployment in the Azure portal. The deployment is named 'CreateVm-canonical.0001-com-ubuntu-server-jammy-2-20251004180030' and is in a 'Complete' state. The 'Deployment details' table lists three resources: 'fitdash-vm2' (Microsoft.Compute/virtualMachines), 'fitdash-vm2400' (Microsoft.Network/networkInterfaces), and 'fitdash-vm2-nsg' (Microsoft.Network/networkSecurityGroups). All resources are in a 'Created' or 'OK' state. The 'Next steps' section includes links to 'Go to resource' and 'Create another VM'. The right sidebar contains links for 'Cost Management', 'Microsoft Defender for Cloud', 'Free Microsoft tutorials', and 'Work with an expert'.

Overview

Your deployment is complete

Deployment name: CreateVm-canonical.0001-com-ubuntu-server-j... Start time: 10/4/2025, 6:04:09 PM
Subscription: Azure subscription 1 Correlation ID: 58f9b0f0-b6b9-46a0-8399-70c9c14041a

Resource group: fitdash

Deployment details

Resource	Type	Status	Operation details
fitdash-vm2	Microsoft.Compute/virtualMachines	Created	Operation details
fitdash-vm2400	Microsoft.Network/networkInterfaces	Created	Operation details
fitdash-vm2-nsg	Microsoft.Network/networkSecurityGroups	OK	Operation details

Next steps

Setup auto-shutdown Recommended

Monitor VM health, performance and network dependencies Recommended

Run a script inside the virtual machine Recommended

[Go to resource](#) [Create another VM](#)

Cost Management
Get notified to stay within your budget and prevent unexpected charges on your bill. [Set up cost alerts](#)

Microsoft Defender for Cloud
Secure your apps and infrastructure [Go to Microsoft Defender for Cloud](#)

Free Microsoft tutorials
[Start learning today](#)

Work with an expert
Azure experts are service provider partners who can help manage your assets on Azure

Bastion

Bastion helped create a secure tunnel between my local browser and the VM inside the private network which was safer than opening SSH to the public internet since no ports or IPs were being exposed. Although, I had realized that Bastion had a continuous and hourly cost so I made sure to delete Bastion and re-deploy it when I was in need of it to minimize costs.

Microsoft Azure Upgrade Search resources, services, and docs (G+)

Home > fitdash-vm1

fitdash-vm1 | Bastion ☆ ...

Virtual machine

Search

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Resource visualizer

Connect

Connect

Bastion

Networking

Settings

Availability + scale

Size

Availability + scaling

Security

Identity

Microsoft Defender for Cloud

Backup + disaster recovery

Azure Bastion protects your virtual machines by secure and seamless RDP & SSH connectivity without the need to expose them through public IP addresses. [Learn more](#)

Using Bastion: **fitdash-vnet-bastion**

Provisioning State: **Succeeded**

Please enter username and password to your virtual machine to connect using Bastion.

Connection Settings

Protocol * ⓘ

☐ RDP ☒ SSH

Port ⓘ

22

Authentication Type ⓘ

SSH Private Key from Local File

Username ⓘ

azureuser

Local File ⓘ

"fitdash-vm1-key.pem"

Advanced

☒ Open in new browser tab

Connect

[Tell us what you think of the Bastion experience](#)

Microsoft Azure Upgrade Search resources, services, and docs (G+)

Copilot

Home >

fitdash-vnet-bastion ☆ ...

Bastion

Search

Delete Refresh

For advanced security and session management needs, you can easily upgrade to Bastion Premium. [Click here to upgrade](#)

JSON View

Essentials

Resource group (moved) : [fitdash](#)

Location : East US

Subscription (moved) : [Azure subscription 1](#)

Subscription ID : 8c187024-9a20-4a71-9b76-29e2893acd73

Tags (edit) : [Add tags](#)

Virtual network/subnet : [fitdash-vnet/AzureBastionSubnet](#)

Public DNS name	: bst-84f08f9-7bf9-4f0e-8e62-8174697083dd.bastion.azure.com
Public IPv4 address	: fitdash-vnet-ipv4
Tier	: Standard
Provisioning state	: Succeeded
Availability zone	: None

Tutorials Sessions

Free training from Microsoft

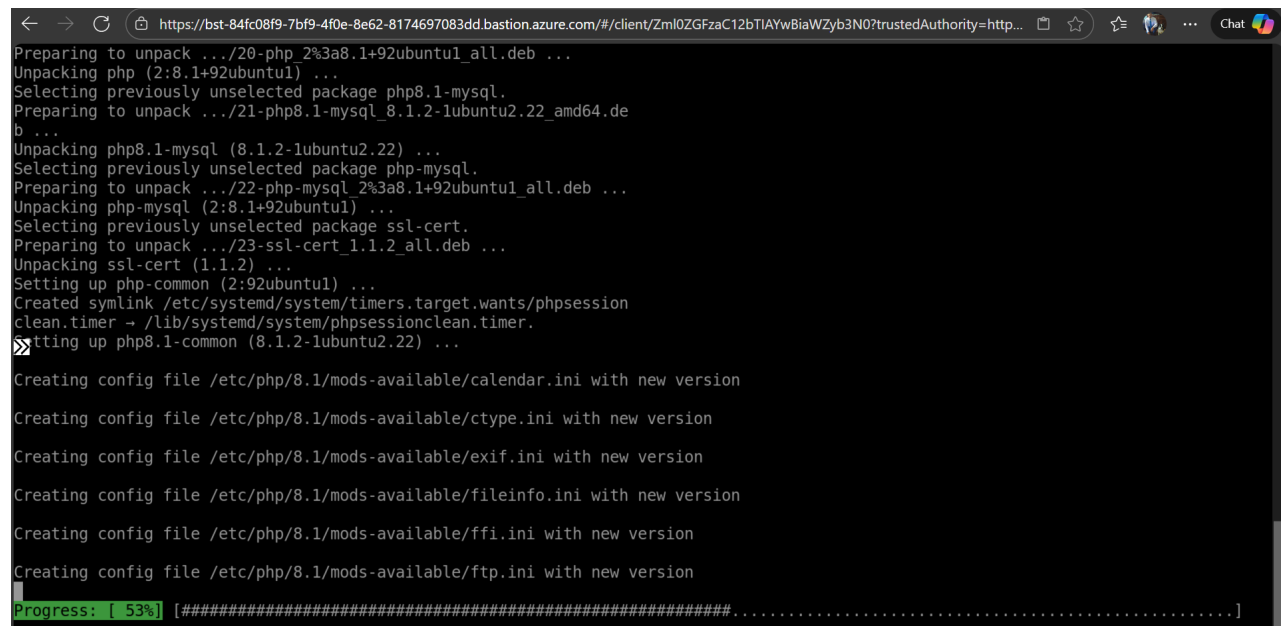
Connect to virtual machines through the Azure portal by using Azure Bastion
8 units • 40 min
Deploy Azure Bastion to securely connect to Azure virtual machines directly within the Azure portal. Start "Connect to virtual machines" training. [Learn more](#)

Configure and manage virtual networks for Azure administrators
15 units • 13 hr 39 min
Learn how to configure and manage a range of Azure network capabilities including connectivity services and network monitoring services. Start "Configure and manage virtual networks" training. [Learn more](#)

LAMP Stack Installation

I installed Apache, PHP, and SQL Server drivers to host my FitDash web app. One lesson I learned is that even simple settings like missing PHP extensions can cause my web page to show an HTTP 500 error. I ran the command “sudo apt update && sudo apt upgrade -y sudo apt install apache2 mysql-client php php-mysql libapache2-mod-php -y” where Apache served as the web server and PHP handled application logic. MySQL client was also installed so I could test connections to the Azure SQL Database. I created a directory under /var/www/html/fitness and uploaded my app files. The app included forms for workout logging, food tracking, and displaying calorie balances.

```
azureuser@fitdash-vm2:~$ sudo apt update
sudo apt install -y apache2 php php-mysql libapache2-mod-php
sudo systemctl enable apache2
sudo systemctl start apache2
Hit:1 http://azure.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://azure.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://azure.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu jammy-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
43 packages can be upgraded. Run 'apt list --upgradable' to see them.
azureuser@fitdash-vm2:~$
```



```
Preparing to unpack .../20-php 2%3a8.1+92ubuntu1_all.deb ...
Unpacking php (2:8.1+92ubuntu1) ...
Selecting previously unselected package php8.1-mysql.
Preparing to unpack .../21-php8.1-mysql_8.1.2-1ubuntu2.22_amd64.de
b ...
Unpacking php8.1-mysql (8.1.2-1ubuntu2.22) ...
Selecting previously unselected package php-mysql.
Preparing to unpack .../22-php-mysql 2%3a8.1+92ubuntu1_all.deb ...
Unpacking php-mysql (2:8.1+92ubuntu1) ...
Selecting previously unselected package ssl-cert.
Preparing to unpack .../23-ssl-cert_1.1.2_all.deb ...
Unpacking ssl-cert (1.1.2) ...
Setting up php-common (2:92ubuntu1) ...
Created symlink /etc/systemd/system/timers.target.wants/phpsession
clean.timer → /lib/systemd/system/phpsessionclean.timer.
Setting up php8.1-common (8.1.2-1ubuntu2.22) ...
Creating config file /etc/php/8.1/mods-available/calendar.ini with new version
Creating config file /etc/php/8.1/mods-available/ctype.ini with new version
Creating config file /etc/php/8.1/mods-available/exif.ini with new version
Creating config file /etc/php/8.1/mods-available/fileinfo.ini with new version
Creating config file /etc/php/8.1/mods-available/ffi.ini with new version
Creating config file /etc/php/8.1/mods-available/ftp.ini with new version
Progress: [ 53%] [#####.....]
```

```
Last login: Sat Oct 4 23:20:22 2025 from 10.0.4.5
azureuser@fitdash-vm1:~$ curl localhost
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
      * {
        margin: 0px 0px 0px 0px;
        padding: 0px 0px 0px 0px;
      }

      body, html {
        padding: 3px 3px 3px 3px;

        background-color: #D8DBE2;

        font-family: Ubuntu, Verdana, sans-serif;
```

Below you can see the PHP application deployed where I use the Load Balancer’s frontend IP followed by the directory where I host the PHP script.

PHP Version 8.1.2-1ubuntu2.22



System	Linux fitdash-vm1 6.8.0-1031-azure #36-22.04.1-Ubuntu SMP Tue Jul 1 03:54:01 UTC 2025 x86_64
Build Date	Jul 15 2025 12:11:22
Build System	Linux
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/8.1/apache2
Loaded Configuration File	/etc/php/8.1/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/8.1/apache2/conf.d
Additional .ini files parsed	/etc/php/8.1/apache2/conf.d/10-mysqlnd.ini, /etc/php/8.1/apache2/conf.d/10-opcache.ini, /etc/php/8.1/apache2/conf.d/10-pdo.ini, /etc/php/8.1/apache2/conf.d/15-xml.ini, /etc/php/8.1/apache2/conf.d/20-calendar.ini, /etc/php/8.1/apache2/conf.d/20-ctype.ini, /etc/php/8.1/apache2/conf.d/20-dom.ini, /etc/php/8.1/apache2/conf.d/20-exif.ini, /etc/php/8.1/apache2/conf.d/20-fileinfo.ini, /etc/php/8.1/apache2/conf.d/20-ftp.ini, /etc/php/8.1/apache2/conf.d/20-gettext.ini, /etc/php/8.1/apache2/conf.d/20-iconv.ini, /etc/php/8.1/apache2/conf.d/20-mbstring.ini, /etc/php/8.1/apache2/conf.d/20-mysql.ini, /etc/php/8.1/apache2/conf.d/20-pdo_mysql.ini, /etc/php/8.1/apache2/conf.d/20-phar.ini, /etc/php/8.1/apache2/conf.d/20-posix.ini, /etc/php/8.1/apache2/conf.d/20-readline.ini, /etc/php/8.1/apache2/conf.d/20-shmop.ini, /etc/php/8.1/apache2/conf.d/20-simplexml.ini, /etc/php/8.1/apache2/conf.d/20-sockets.ini, /etc/php/8.1/apache2/conf.d/20-sqlsrv.ini, /etc/php/8.1/apache2/conf.d/20-sysvmsg.ini, /etc/php/8.1/apache2/conf.d/20-sysvsem.ini, /etc/php/8.1/apache2/conf.d/20-sysvshm.ini, /etc/php/8.1/apache2/conf.d/20-tokenizer.ini, /etc/php/8.1/apache2/conf.d/20-xmlreader.ini, /etc/php/8.1/apache2/conf.d/20-xmlwriter.ini, /etc/php/8.1/apache2/conf.d/20-xsl.ini, /etc/php/8.1/apache2/conf.d/30-pdo_sqlsrv.ini
PHP API	20210902
PHP Extension	20210902
Zend Extension	420210902
Zend Extension Build	API420210902,NTS

 http://4.156.98.250/index.php

FitDash (demo)

Add Workout

Exercise:

Duration (minutes):

Calories:

Workouts

ID	Exercise	Duration	Calories
1	Running	30	300

Upload Image (to Blob via SAS)

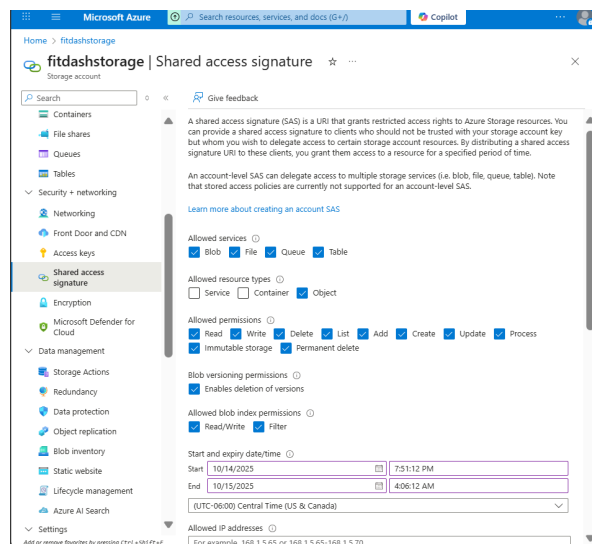
Choose image: No file selected.

Storage

The Azure Storage Account (fitdashstorage) hosts the Blob container (images) which stores application media such as workouts, profile images, etc. Using Blob Storage separates the static assets from compute instances and it ensures durability and reduces the VM storage usage. Each file is accessible via a unique URL, and access can be secured through SAS tokens or private networking.

SAS Token Setup

SAS (Shared Access Signature) token is a secure key that gives temporary, limited access to your Azure Blob Storage account. The PHP code needs authorization from the Azure Blob storage and the best way to allow access is by generating an SAS token to provide temporary access. So instead of your PHP code using permanent credentials, it passes the SAS token in the upload request URL.



Fitdashstorage container for images

In my FitDash project, the storage account “fitdashstorage” acts as the repository for all uploaded content coming from users through your PHP website. Using Blob Storage showed me how Azure can handle both structured and unstructured data. I also liked how simple it was to manage through the Portal without any CLI commands.

Create a storage account

Basics

- Subscription: Azure subscription 1
- Resource group: fitdash
- Location: East US
- Storage account name: fitdashstorage
- Preferred storage type: Azure Blob Storage or Azure Data Lake Storage Gen 2
- Performance: Standard
- Replication: Locally-redundant storage (LRS)

Advanced

- Enable hierarchical namespace: Disabled
- Enable HTTP: Disabled
- Enable network file system (v): Disabled
- Allow cross-tenant replication: Disabled
- Access tier: Hot
- Enable large file shares: Enabled

Security

- Secure transfer: Enabled
- Blob anonymous access: Disabled
- Allow storage account key access: Enabled
- Default to Microsoft Entra authorization in: Disabled

Previous Next **Create**

fitdashstorage_1760229479744 | Overview

Deployment

Your deployment is complete

Deployment name: fitdashstorage_1760229479744
Subscription: Azure subscription 1
Resource group: fitdash

Start time: 10/11/2025, 7:39:01 PM
Correlation ID: 9c8baa1f-d114-4f00-a77b-7a08d81da44f

Resource	Type	Status	Operation details
fitdashstorage/default	Microsoft.Storage/storageAccounts/fileservices	OK	Operation details
fitdashstorage/default	Microsoft.Storage/storageAccounts/blobServices	OK	Operation details
fitdashstorage	Microsoft.Storage/storageAccounts	OK	Operation details

Next steps

[Go to resource](#)

Cost Management
Get notified to stay within your budget and prevent unexpected charges on your bill. [Set up cost alerts](#)

Microsoft Defender for Cloud
Secure your apps and infrastructure. [Go to Microsoft Defender for Cloud](#)

Free Microsoft tutorials
[Start learning today](#)

Blob Storage provided the media files that were embedded in workout and meal logs, while the SQL Database stored the metadata and links. This demonstrated how storage and database services can complement each other in a real-world web app. When a user uploads a picture through the FitDash form, your PHP code sends it to the blob container instead of saving it locally on the VM.

fitdashstorage | Containers

Storage account

Containers

- File shares
- Queues
- Databases

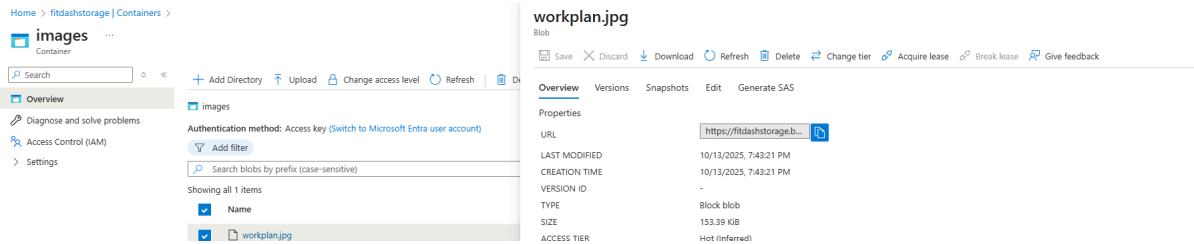
New container

Name: Images

Anonymous access level: Private (no anonymous access)

Create

The images are stored safely and backed up automatically in Azure's cloud. I can easily serve those images on your FitDash site using their unique blob URLs, in example below:
<https://fitdashstorage.blob.core.windows.net/images/workplan.jpg> (link discontinued)

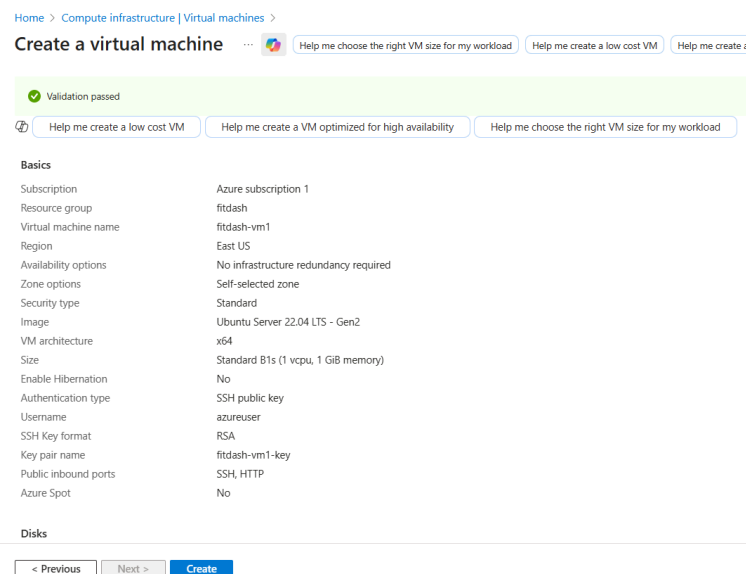


Compute

For my FitDash web application, I created two Ubuntu virtual machines that act as my main web servers. These servers run Apache and PHP, which host the FitDash website that users can access through a browser. At first, I connected to these VMs using Azure Bastion, but I quickly learned that Bastion costs a lot of money when left running. As a workaround I realized I could save money by deleting Bastion and then re-deploying it when absolutely necessary. But after some research I could have enabled Just-in-Time access in Azure Defender or connected using SSH with a secure key pair. This change saved me more than half of my credit usage each week.

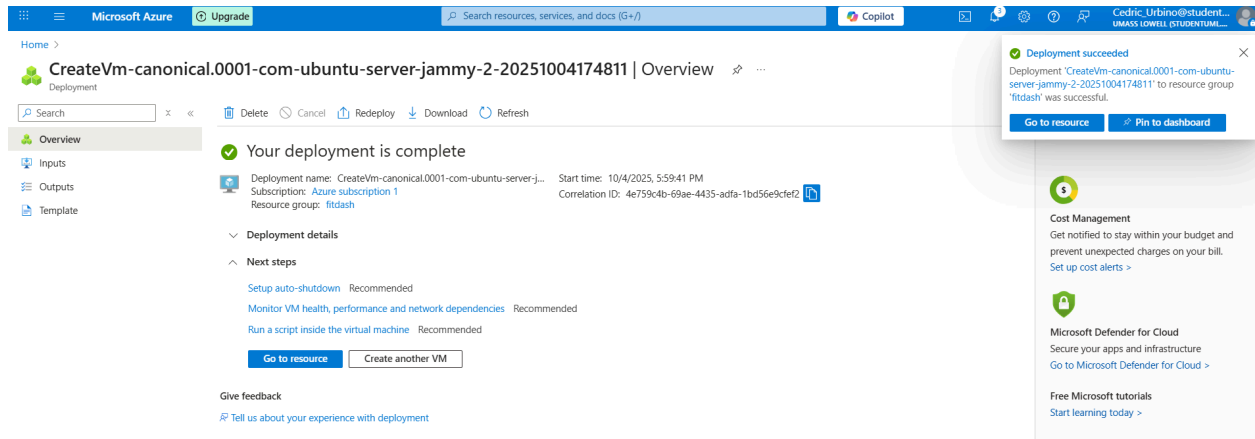
Virtual Machines

I deployed two Ubuntu virtual machines to host the Fitness Tracker Dashboard web application. I chose the Standard_B1s size because it is lightweight (1 vCPU, 1 GB RAM) and inexpensive, making it ideal for a class project.



I also learned to use a Load Balancer to evenly distribute web traffic between both virtual machines. By doing so, even if one VM goes offline, the second one automatically handles the

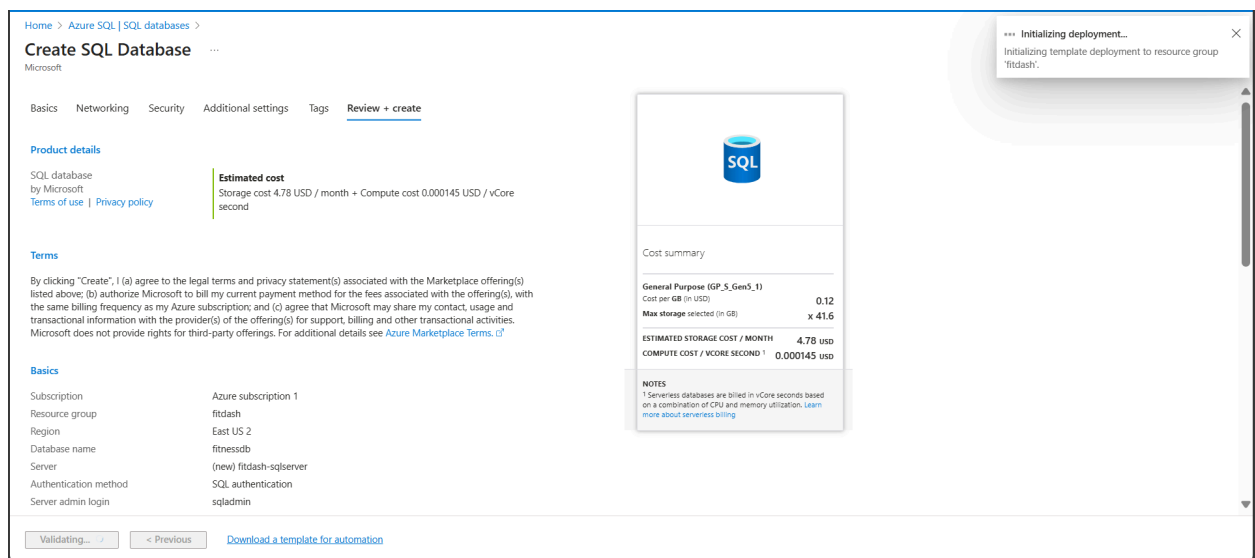
incoming requests. This is called high availability and it's a key concept in cloud computing. Through this setup, I learned how virtual machines serve as the foundation for web hosting and how Azure simplifies management by allowing snapshots, scaling, and remote control.

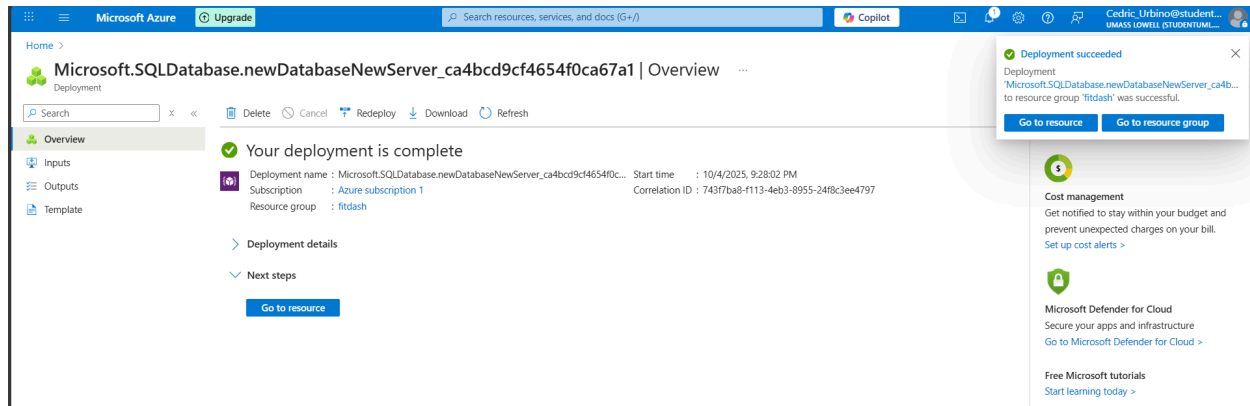


The VMs hosted the web app, connected to the Azure SQL Database for workout and food logs, pulled images from Blob Storage, and were load-balanced for availability.

Database

The database stores all the FitDash user information, workout entries, and meal logs basically all the structured data that powers the app's tracking and reporting features. I had made the mistake of making my SQL Database to a serverless standard so I had to scale my Azure SQL Database down to basic because it was quickly becoming one of my biggest costs. It had already used up \$30 of my credit after only a few days of use. After switching it to the Basic tier, it now costs around \$5 per month, which is much more manageable for this project.





Scale database in progress Running X

Scaling from General Purpose - Serverless: Standard-series (Gen5), 1 vCore, 32 GB storage, zone redundant disabled to Basic: 2 GB storage for database: fitnessdb.

a few seconds ago

Scale database success X

Successfully scaled from General Purpose - Serverless: Standard-series (Gen5), 1 vCore, 32 GB storage, zone redundant disabled to Basic: 2 GB storage for database: fitnessdb.

a few seconds ago

Fitnessdb, tables (Users, Workouts, Meals)

I set up a serverless Azure SQL Database named fitnessdb. This database stores all FitDash information such as user profiles, workouts, calories burned, and meal logs. I chose the serverless compute tier because it automatically pauses when idle, saving costs, and resumes when new queries arrive. However, I discovered it still charges a small amount for storage and a minimum runtime.

At first, I made a mistake by accidentally creating my tables inside the master database instead of my custom fitnessdb schema. This caused permission errors and foreign key issues when I tried to create relationships between the Users, Workouts, and Meals tables. After realizing the error, I recreated everything correctly in the right database. I used SQLCMD from my VM to build the tables and run queries, the credentials for my server are “redacted”.

Initially, I kept trying to use MySQL commands, which didn’t work since Azure SQL uses T-SQL. Switching to the correct SQL syntax and drivers finally allowed the PHP web app to store and read data correctly. This process taught me how important it is to match the right database tools

to the correct engine. I also learned that Azure SQL provides strong built-in security features like firewalls and transparent data encryption without extra setup.

So to test my Database I used this:

```
azureuser@fitdash-vm1:~$ echo 'export PATH="$PATH:/opt/mssql-tools /bin"' >> ~/.bashrc
```

```
azureuser@fitdash-vm1:~$ source ~/.bashrc azureuser@fitdash-vm1:~$ sqlcmd -S
```

```
fitdash-sqlserver.database.windows.net -U sqladmin -P 'Sqlpassword1!'
```

Which then allowed me create the database and modify table entries within my VM.

[Home](#) > [Azure SQL | SQL databases](#) > [Create SQL Database](#) >

Create SQL Database Server

Microsoft

Server name * ✓
 .database.windows.net

Location * ✓

Authentication

 Azure Active Directory (Azure AD) is now Microsoft Entra ID. [Learn more](#) 

Select your preferred authentication methods for accessing this server. Create a server admin login and password to access your server with SQL authentication, select only Microsoft Entra authentication [Learn more](#)  using an existing Microsoft Entra user, group, or application as Microsoft Entra admin [Learn more](#) , or select both SQL and Microsoft Entra authentication.

Authentication method

☐ Use Microsoft Entra-only authentication

☐ Use both SQL and Microsoft Entra authentication

☒ Use SQL authentication

Server admin login * ✓

Password * ✓

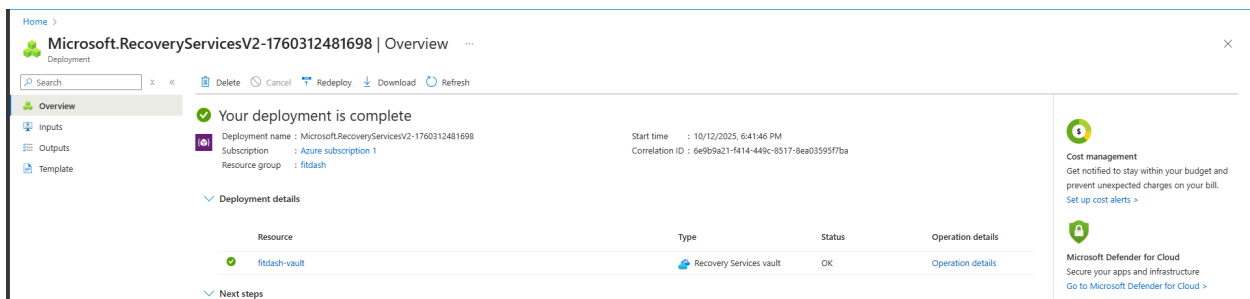
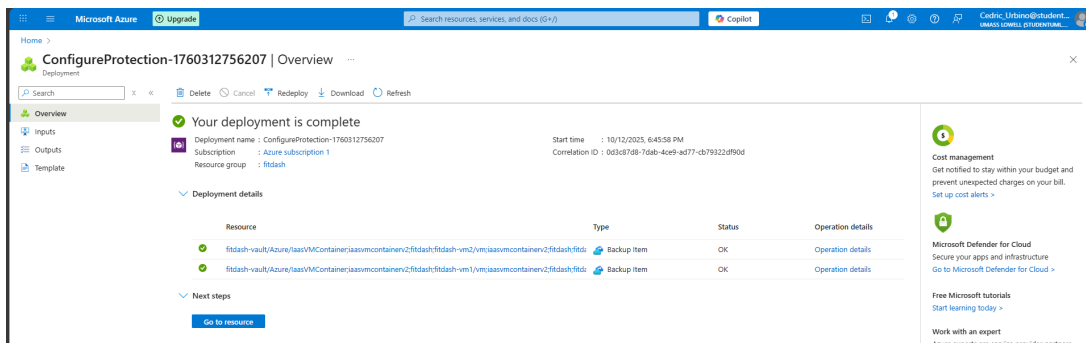
Confirm password * ✓

OK

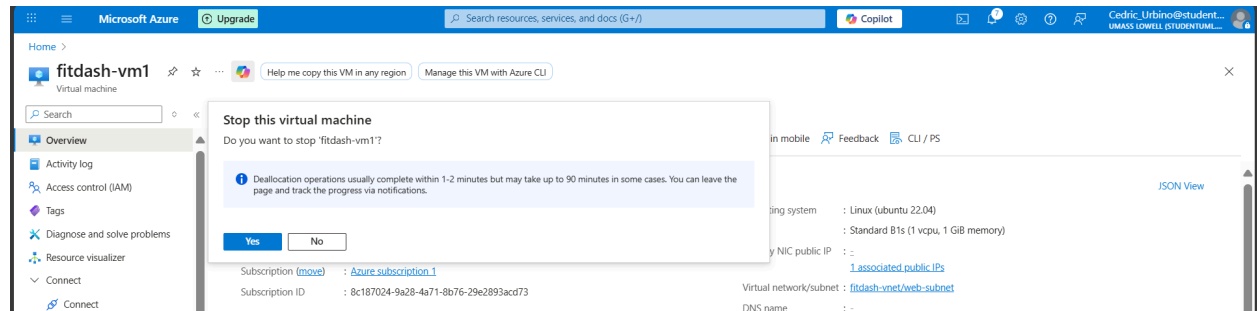
I did have to update the SQL database to East US 2 which is the sister region to East US and it should still work but there might be very minute latency.

Backup (Recovery Services Vault)

The Recovery Vault allows both snapshot backups and application-consistent backups, which capture all active data without corrupting it. I simulated a test restore by creating a recovery point for one VM, and it worked as expected. This feature gave me a sense of security knowing that even if I broke something while testing, I could roll back easily.



Here in my testing, I had stopped one VM and the Load Balancer automatically routed traffic to the healthy VM which in this case was “fitdash-vm2”

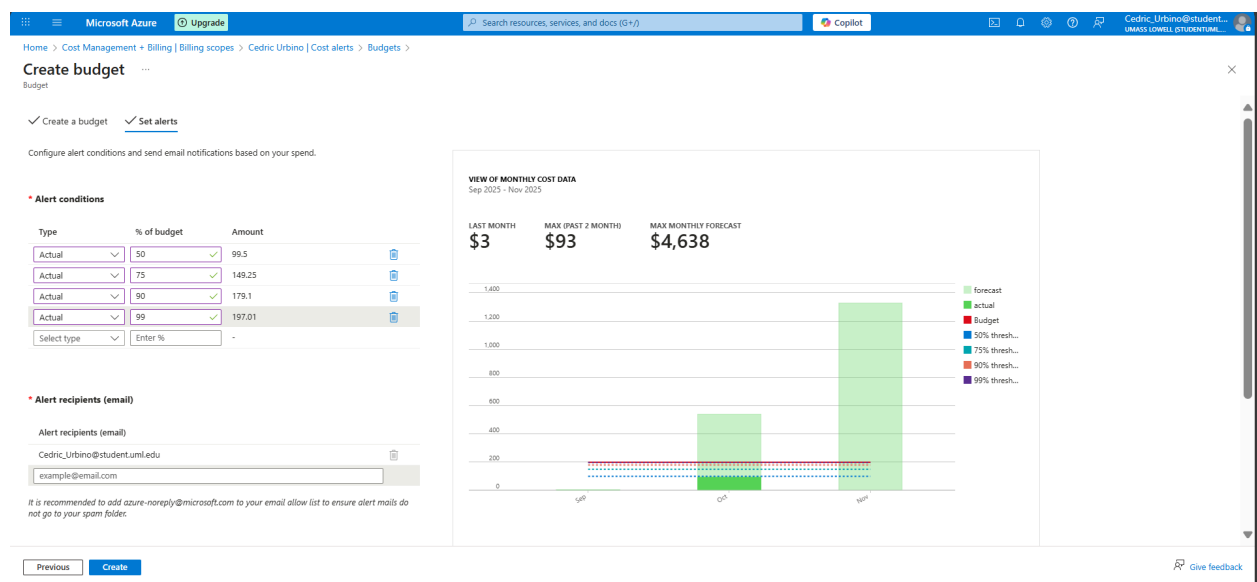


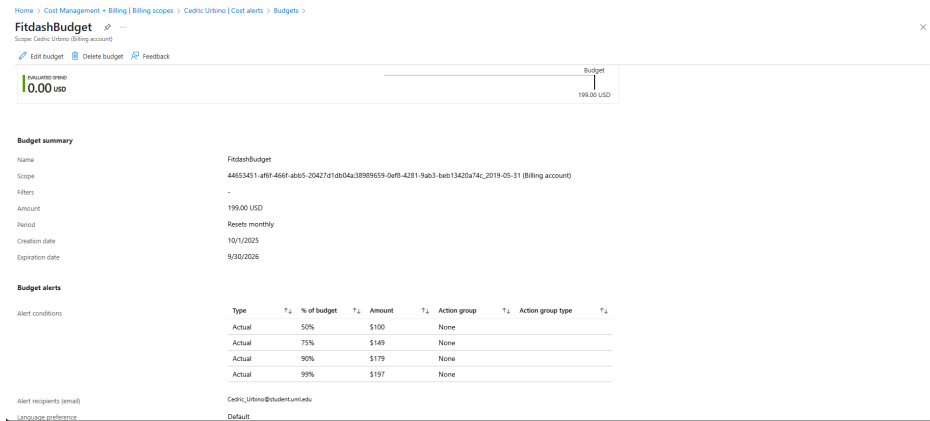
Monitoring and Billing

Cost management was one of the most important lessons from this project. I created a budget alert called FitdashBudget to track my free credit balance. Initially, I set thresholds too high, which meant I didn't get alerts until I had already spent most of my budget. I later adjusted them to 50%, 75%, 90%, and 99% of the \$199 credit limit.

I discovered that Azure Bastion and SQL Database were consuming the most funds, followed by the VMs. I stopped Bastion and switched to SSH at one point, reduced VM uptime, and kept SQL in serverless mode. I also learned that shutting down VMs when not in use immediately stops hourly billing. Setting strict cost alerts taught me real budgeting habits where somethings you don't expect are often overlooked in technical work.

By the end, I had about \$100 of my credit left, which I preserved by managing resources carefully. This experience made me realize that technical skill is only one part of cloud projects as cost control is equally important.





Azure Monitor Alerts

Since my Azure account includes \$199.87 of free credit that expires in 21 days, I created a budget in Azure Cost Management to ensure I stay within that limit. The budget is configured for the subscription and is set to a total of \$200 for the 21-day period.

The budget provides me with plenty of room to experiment without running out of credit. Even with two VMs running 24/7, a database, blob storage, and networking, the total cost is still well below \$200. This ensures I can safely complete the project while also testing backups, monitoring, and alerts without worrying about accidental overspending.

The purpose of my Action Group and Alert Rule below is to help me monitor and respond to important changes or issues in your FitDash Azure environment. The Alert Rule continuously watches for certain conditions. For example, when the VM CPU usage goes above 80% or when the SQL Database DTU gets too high the alert is triggered automatically.

Home > Monitor | Alerts >

Create an alert rule

Scope Condition Actions Details Tags Review + create

Product details

Metric alert rule

1 Condition

[Terms of use](#) | [Privacy statement](#)

Total pricing

0.10 USD/month

[Pricing](#)

Scope

Scope level

Resource

Subscription

Azure subscription 1 > fitdash > fitdash-vm1

Condition

Signal name

Operator

Aggregation type

Threshold value

Lookback period

Check every

Percentage CPU

Greater than

Average

80

5 minutes

1 minute

Actions

Action group name

fitdash-alerts

Contain actions

1 Email

Details

Create Previous

Microsoft Azure Search resources, services, and docs (G+/) Copilot

Home > Monitor | Alerts >

Create an alert rule

Scope Condition Actions Details Tags Review + create

Product details

Metric alert rule	Total pricing
1 Condition	0.10 USD/month
Terms of use Privacy statement	Pricing

Scope

Scope level	Subscription
Resource	Azure subscription 1 > fitdash > fitdash-sqlserver/fitnessdb

Condition

Signal name	dtu_consumption_percent
Operator	Greater than
Aggregation type	Average
Threshold value	80
Lookback period	5 minutes
Check every	1 minute

Actions

Action group name	Contain actions
fitdash-alerts	1 Email

Create Previous

✓ **Alert rule created** ✕

Alert rule fitdash-SQL-UsageAlert successfully created. It might take a few minutes for changes to be shown.

a few seconds ago

✓ **Alert rule created** ✕

Alert rule fitdash-HighCPUAlert successfully created. It might take a few minutes for changes to be shown.

2 minutes ago

The Action Group defines what happens next since it is the set of actions that Azure takes when an alert is triggered. My Action Group “fitdash-alerts” sends email notifications so that I am instantly informed about performance issues or cost spikes.

Microsoft Azure

Upgrade

Search resources, services, and docs (G+/)

[Home](#) > [Monitor](#) | [Alerts](#) > [Create an alert rule](#) >

Create action group

Basics

Notifications

Actions

Tags

Review + create

This is a summary of your action group. Please review to ensure the information is correct and consider [Azure Monitoring Pricing](#) and the [Azure Privacy Statement](#).

Basics

Subscription

Azure subscription 1

Resource group

fitdash

Region

global

Action group name

fitdash-alerts

Display name

FitdashAlert

Notifications

Notification type

Name

Selected

Email/SMS message/Push/Voice

Fitdash Alert

Email

Actions

None

Tags

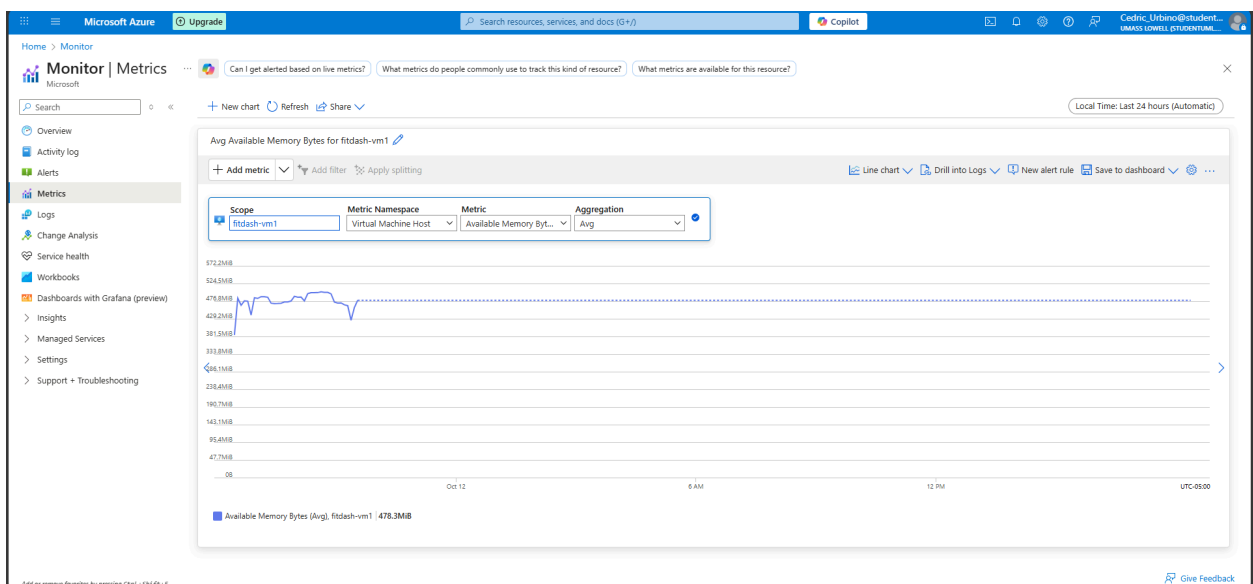
None

Create

Previous

I learned that monitoring can track multiple services in one place. This made me realize how important it is to have visibility into performance and costs.

This metric graph is tracking the “Available Memory Bytes (Avg)” for my virtual machine fitdash-vm1. The blue line represents how much free memory (RAM) the VM has available over time. The average available memory shown is around 478 MB, meaning that the VM is not under heavy memory pressure. This is a prime example of how useful monitoring metrics is to determine the overall status and health of my system.



Add or remove favorites by pressing Ctrl+Shift+F

Give Feedback

Conclusion

The FitDash project brought together multiple Azure services into one functional, scalable, and cost efficient fitness tracking system. Through it, I learned the full lifecycle of deploying a cloud application starting from the compute setup and networking to storage, identity, and monitoring.

Each part of the project reinforced a different real-world skill. As the Virtual Machines taught me how to host and troubleshoot web servers. SQL Database showed me how to structure and protect data. Blob Storage handled images easily and securely. The Load Balancer improved reliability and performance. The Recovery Vault gave me peace of mind with backups. Azure Monitor and Billing alerts kept me proactive and budget-aware.

FitDash demonstrates how a modern cloud platform supports both users and developers. The system lets users log workouts and meals while the images are stored securely, and all components are protected by backups and alerts.

The biggest takeaway is how important service interaction is. If I expand this project in the future, I'd like to add user authentication with Azure AD B2C and possibly integrate APIs for food nutrition lookup or incorporate the use of Azure's Functions to use with some kind of calorie tracker.

Along the way, I made several mistakes like losing internet access after connecting the Load Balancer, connecting to the wrong SQL database, and causing my app to crash due to missing PHP extensions but each issue helped me understand Azure better. I also learned the importance of cost management, since running services continuously can burn through credits quickly. By the end, I had a fully deployed and monitored cloud system that mirrors what real companies use for scalable web applications.

References

Create a virtual network (Azure Portal)

<https://learn.microsoft.com/en-us/azure/virtual-network/quickstart-create-virtual-network?tabs=portal>

Create a Load Balancer (Azure Portal)

<https://learn.microsoft.com/en-us/azure/load-balancer/quickstart-load-balancer-standard-public-portal?tabs=option-1-create-load-balancer-standard>

Create a Virtual Machine (Ubuntu)

<https://learn.microsoft.com/en-us/azure/virtual-machines/linux/quick-create-portal?tabs=ubuntu>

Create Azure SQL Database (Portal)

<https://learn.microsoft.com/en-us/azure/azure-sql/database/single-database-create-quickstart?tabs=azure-portal>

Azure Blob Storage – Create a Storage Account

<https://learn.microsoft.com/en-us/azure/storage/common/storage-account-create?tabs=azure-portal>

Create Azure Monitor Alerts

<https://learn.microsoft.com/en-us/azure/azure-monitor/alerts/alerts-overview>

Azure Recovery Services Vault

<https://learn.microsoft.com/en-us/azure/backup/backup-create-rs-vault>

Azure AD B2C

<https://learn.microsoft.com/en-us/azure/active-directory-b2c/tutorial-create-tenant>

Azure Functions (Serverless Compute)

<https://learn.microsoft.com/en-us/azure/azure-functions/functions-create-function-app-portal>

Azure Cost Management Alerts

<https://learn.microsoft.com/en-us/azure/cost-management-billing/costs/cost-mgt-alerts-monitor-usage-spending>